Scaling Conjugate Gradients with MPI and OpenMP

Decomposition, communication hiding, memory bandwidth and GPU offload in a matrix-free 3D Poisson solver

WU YUHAN (A0351706H) ZUO DEWEI (A0352507H) ZHU CHEN (A0357394N)

DSA5208 · Project 3

Abstract

The Conjugate Gradient (CG) method is the standard solver for the symmetric positive definite systems produced by elliptic partial differential equations, and it is notoriously hard to scale: every iteration needs a nearest-neighbour exchange, two global reductions and a full sweep through memory. We build a matrix-free CG solver for the 3D Poisson equation in three versions—serial, hybrid MPI + OpenMP, and MPI + OpenMP target offload to GPUs—in which domain decomposition, halo exchange, reduction strategy and the split between processes and threads are run-time switches, and measure each choice on the NUS Vanda cluster from one core to 256 cores on 8 nodes and on an NVIDIA A40. The solver reproduces the serial solution to 3.8×10^{-15} in 180 CPU and 8 GPU configurations and converges at the theoretical second order. Inside a socket CG scales exactly as far as memory bandwidth does: 32 cores reach a speedup of 9.1 and 77% of the STREAM bandwidth. Across nodes every socket brings its own memory, and 256 cores run a 512^3 iteration in 9.92 ms, a speedup of 55.1 with 76% efficiency relative to one node (83% for 768^3) and 80% weak-scaling efficiency on 1.8 billion unknowns. At that scale the global reductions, not the halo messages, cost most: they take 28% of an iteration, almost all of it waiting at a barrier. Merging the two reductions saves 5%, pipelining them costs more memory traffic than it hides, and one A40 matches 2.4 CPU sockets.

1. Introduction

Many simulation codes spend most of their run time solving a single kind of linear system. The pressure projection of incompressible flow solvers, the electrostatic potential in plasma and molecular dynamics, the gravitational potential in astrophysics and graph-Laplacian systems in semi-supervised learning and spectral clustering all reduce to a Poisson-type equation that has to be solved again at every time step or outer iteration. Discretising $-\nabla^2 u = f$ on a cube with N grid points per direction yields N^3 unknowns, and refining the grid makes the problem harder twice over: the system grows like N^3, and the number of iterations of a Krylov solver grows like N, so the total work grows like N^4. A 512^3 grid already holds 134 million unknowns, needs about two thousand CG iterations and takes 19.8 minutes on one core. Solving such problems in reasonable time and memory requires distributing them across many nodes.

The Conjugate Gradient method is the method of choice for these symmetric positive definite systems, and it is also a standard stress test for parallel computers: the HPCG benchmark [4], which complements LINPACK in the TOP500 ranking, is built around it because supercomputers reach only a few percent of their peak performance on it. The reason lies in the structure of a single iteration. Its matrix–vector product couples every unknown to its grid neighbours, so a distributed implementation has to exchange the boundary layers (*halos*) of every subdomain in every iteration. Its two inner products couple every unknown to every other one; each requires a global reduction that acts as a barrier across all processes and whose latency grows with the number of processes P. Since every iteration needs the scalars produced by the previous one, none of this communication can be batched or deferred across iterations. What remains after communication—the stencil and the vector updates—performs about 17 floating-point operations per 88 bytes of memory traffic, so even the perfectly parallel local work is limited by the memory bandwidth that all cores of a socket share.

Each of these obstacles admits several engineering answers, and the answers interact. A three-dimensional block decomposition minimises the halo volume but sends more, smaller and strided messages than a slab decomposition. Non-blocking communication can hide the halo exchange behind interior computation, but only if the MPI library actually progresses the messages and the deferred boundary work does not destroy cache locality. Algebraic reformulations of CG merge the two reductions into one or overlap the reduction with the matrix–vector product, at the price of extra vectors, extra memory traffic and weaker numerical stability. OpenMP threads inside each MPI process cut the number of messages and of reduction participants, at the price of fork–join overhead and threads that idle while the master thread communicates. A GPU changes the balance again: it offers several times the memory bandwidth of a CPU socket, but every value that crosses the PCIe bus to reach the network costs more than it costs on the host.

This project implements a serial reference solver, a hybrid MPI + OpenMP solver and an MPI + OpenMP offload solver in C, with each design choice a run-time switch so that its effect is measured in isolation on an identical problem. We validate the parallel solvers point by point against the serial one and against an analytical solution, derive a performance model from memory traffic and communication volume, and instrument the solvers with per-phase timers and byte counters that explain the measured scaling rather than merely report it. Section 2 introduces the problem and the algorithm, section 3 develops the parallel design and the performance model, section 4 validates the implementation and section 5 analyses its performance from one core to 8 nodes and on a GPU.

2. The model problem and the Conjugate Gradient method

We solve the Poisson equation $-\nabla^2 u = f$ on the unit cube $\Omega = (0,1)^3$ for an unknown function $u : \Omega \to \mathbb{R}$ and a given source f, with the homogeneous Dirichlet condition $u = 0$ on the boundary $\partial\Omega$. A uniform grid with N interior points per direction has spacing $h = 1/(N+1)$ and grid points $(x_i, y_j, z_k) = (ih, jh, kh)$ for $i, j, k \in \{1, \dots, N\}$; we write u_{ijk} for the approximation of $u(x_i, y_j, z_k)$. Replacing each second derivative by the centred difference $(u_{i-1} - 2u_i + u_{i+1})/h^2$ and multiplying by h^2 gives the linear system $A\mathbf{u} = \mathbf{b}$ with

$$\begin{aligned} (A\mathbf{u})_{ijk} &= 6u_{ijk} - u_{i-1,j,k} - u_{i+1,j,k} - u_{i,j-1,k} - u_{i,j+1,k} - u_{i,j,k-1} - u_{i,j,k+1}, \\ b_{ijk} &= h^2 f(x_i, y_j, z_k), \end{aligned} \tag{1}$$

where values with an index equal to 0 or $N+1$ are boundary values and therefore zero. The matrix $A \in \mathbb{R}^{N^3 \times N^3}$ is symmetric positive definite, has $7N^3 - 6N^2$ nonzeros and condition number $\kappa(A) \approx 4/(\pi^2 h^2)$. We never assemble A: it is applied as the 7-point stencil (1) (*matrix-free*), so memory consumption and memory traffic are entirely those of the vectors.

To test the solver we prescribe the exact solution $u(x,y,z) = g(x)\,g(y)\,g(z)$ with $g(t) = t(1-t)e^t$, which vanishes on $\partial\Omega$, and compute the matching source $f = -\nabla^2 u$ analytically using $g''(t) = -t(3+t)e^t$. The common choice $\sin(\pi x)\sin(\pi y)\sin(\pi z)$ would be useless here: it is an eigenvector of the discrete operator, CG converges on it in a single iteration, and the test would hide most bugs. For our u the discrete solution differs from the exact one by a genuine $O(h^2)$ error, which turns the comparison with u into a sharp end-to-end test.

CG [1,6] (algorithm 1) builds the iterates $\mathbf{x}_k$ from the residuals $\mathbf{r}_k = \mathbf{b} - A\mathbf{x}_k$ and the search directions $\mathbf{p}_k$; the step length α_k minimises the A-norm of the error along $\mathbf{p}_k$, and β_{k+1} keeps the new direction A-conjugate to all previous ones. The scalar $\gamma_k = \|\mathbf{r}_k\|^2$ doubles as the stopping criterion with relative tolerance ε. The error contracts by roughly $(\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)$ per iteration, so the number of iterations to reach a fixed ε grows like $\sqrt{\kappa} \propto 1/h \propto N$; our runs need $3.79\,N$ iterations at $\varepsilon = 10^{-8}$ (section 5.1). One iteration consists of one stencil application, two inner products and three vector updates. Per grid point the stencil costs 7 floating-point operations, its inner product 2 and the updates 8, i.e. 17 flops, against 11 loads and stores of 8-byte values (88 bytes, see section 3.6). At 0.2 flop/byte this is an order of magnitude below the balance of current server processors and GPUs, so CG runs at the speed of memory, not of arithmetic.

Algorithm 1 Conjugate Gradients for $A\mathbf{x} = \mathbf{b}$ with $\mathbf{x}_0 = 0$. Framed comments mark the points where a parallel implementation must communicate.

$\mathbf{r}_0 = \mathbf{b}, \quad \mathbf{p}_0 = \mathbf{r}_0, \quad \gamma_0 = \mathbf{r}_0^T \mathbf{r}_0$
for $k = 0, 1, 2, \dots$ **until** $\sqrt{\gamma_k} \le \varepsilon \, \|\mathbf{b}\|$ **do**
  $\mathbf{q}_k = A\mathbf{p}_k$ ▷ halo exchange of $\mathbf{p}_k$, then stencil
  $\alpha_k = \gamma_k / (\mathbf{p}_k^T \mathbf{q}_k)$ ▷ global reduction
  $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k, \quad \mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{q}_k, \quad \gamma_{k+1} = \mathbf{r}_{k+1}^T \mathbf{r}_{k+1}$ ▷ global reduction
  $\beta_{k+1} = \gamma_{k+1}/\gamma_k, \quad \mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_{k+1}\mathbf{p}_k$
end for

3. Parallel design

The parallel solver distributes the grid over P MPI processes and runs T OpenMP threads inside each. This section follows the data through one iteration: how the grid is partitioned, how boundary values travel between processes, how the global reductions are organised and how the threads share the local work. It then describes the GPU version and closes with a cost model that ties these decisions together.

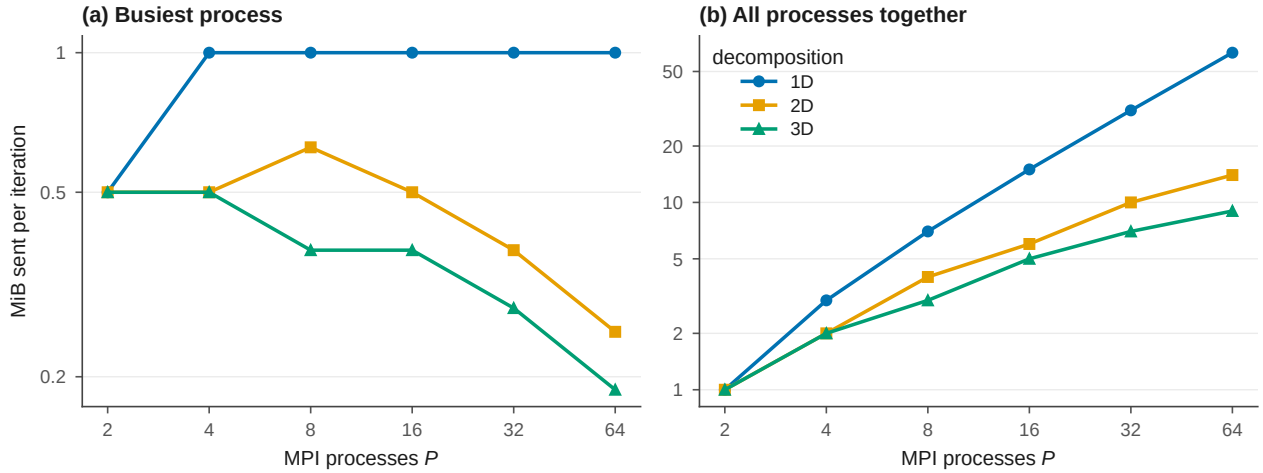


Figure 1: Halo data sent per CG iteration for slab (1D), pencil (2D) and block (3D) decompositions, $N = 256$, measured by the solver's counters. (a) The busiest process, which sets the exchange time on a bandwidth-limited network; (b) the total network load.

3.1 Data distribution

The processes form a Cartesian grid $p_x \times p_y \times p_z = P$ created by `MPI_Dims_create` and `MPI_Cart_create`. The process with coordinates (c_x, c_y, c_z) owns a block of $n_x \times n_y \times n_z$ grid points, where in each direction $n_d = \lfloor N/p_d \rfloor$, plus one for the first $N \bmod p_d$ coordinates; block sizes therefore differ by at most one plane and any P works with any N. Each block is stored with one ghost layer on every side, $(n_x + 2)(n_y + 2)(n_z + 2)$ values with the k index contiguous. Ghost layers on the physical boundary permanently hold the Dirichlet zeros, so the stencil loop contains no branches. The right-hand side is evaluated from one-dimensional tables of g and g'', so no global vector $\mathbf{b}$ is stored.

The switch `-d` selects slabs ($p_y = p_z = 1$), pencils ($p_z = 1$) or blocks (all three factors free). The choice sets how much data each process sends per iteration. For a process in the interior of the process grid and N divisible by the factors, the number of 8-byte values sent per halo exchange is

$$V_{1D} = 2N^2, \qquad V_{2D} = \frac{4N^2}{\sqrt{P}}, \qquad V_{3D} = \frac{6N^2}{P^{2/3}}, \tag{2}$$

carried by $m = 2$, 4 and 6 messages respectively. The slab volume does not decrease with P, so slabs stop scaling once the halo time becomes comparable to the computation, which shrinks like $1/P$; slabs also cap P at N. Blocks send the least data, but their k-faces consist of $n_x n_y$ values separated by a stride of $n_z + 2$, so packing them into a send buffer touches one cache line per value. The solver counts the bytes and messages of every process, and fig. 1 shows the measured values for $N = 256$. At $P = 64$ the busiest process sends 1.00 MiB per iteration with slabs and 0.19 MiB with blocks, a factor of 5.3, while the message count rises from 2 to 6. We use blocks as the baseline.

3.2 Halo exchange

Before each stencil application a process needs the values of the neighbouring blocks adjacent to its faces. Because the 7-point stencil uses no edge or corner neighbours, the three directions are independent and are exchanged together. In the *blocking* mode the solver packs all six faces into buffers within one OpenMP parallel region, performs one `MPI_Sendrecv` per direction and sense, unpacks the received faces into the ghost layers and then applies the stencil to the whole block. Neighbours that do not exist are `MPI_PROC_NULL`, which turns the corresponding transfers into no-ops.

The *overlapped* mode exploits the fact that most points of a block do not need ghost values. It posts `MPI_Irecv` for all faces, packs, posts `MPI_Isend`, applies the stencil to the interior, waits for the messages, unpacks and finally applies the stencil to the remaining boundary shell. The shell is covered by disjoint pieces—two i-planes, two j-rows inside them, two k-columns inside those—so the fused inner product counts every point exactly once. Making this overlap effective took two further steps. The first concerns progress: most MPI libraries move data only inside MPI calls, and large faces use a rendezvous protocol whose handshake cannot complete while the process computes. The solver therefore splits the interior sweep into chunks of i-planes (four by default) and calls `MPI_Testall` between them, which advances the transfers during the computation. The second concerns locality. Our first version deferred the shell layers on all six sides, so with slabs every process handled its $k = 1$ and $k = n_z$

columns as isolated, strided points even though these lie on the physical boundary and need no communication. This cost 10–15% per iteration and made the overlapped mode slower than the blocking one. Restricting the deferred shell to layers that face an actual neighbour removed this penalty; what remains is the cost of the second pass over the shell, which pays off only when messages take longer than that pass (section 5.4).

3.3 Global reductions and three formulations of CG

Classic CG (algorithm 1) performs two `MPI_Allreduce` operations per iteration on a single value each. Nothing useful can run while they are in flight, and with recursive doubling each costs about $\lceil \log_2 P \rceil$ network latencies. Two algebraically equivalent reformulations attack this term.

The Chronopoulos–Gear variant [2] (`-a cg1`) introduces the auxiliary vectors $\mathbf{w}_k = A\mathbf{r}_k$ and $\mathbf{s}_k = A\mathbf{p}_k$. Since $\mathbf{p}_k = \mathbf{r}_k + \beta_k \mathbf{p}_{k-1}$, linearity gives the recurrence $\mathbf{s}_k = \mathbf{w}_k + \beta_k \mathbf{s}_{k-1}$, and the only stencil application per iteration becomes $\mathbf{w}_k = A\mathbf{r}_k$. The denominator $\mathbf{p}_k^T A\mathbf{p}_k$ of α_k can then be expressed through $\gamma_k = \mathbf{r}_k^T \mathbf{r}_k$ and $\delta_k = \mathbf{w}_k^T \mathbf{r}_k$: expanding $\mathbf{p}_k^T A\mathbf{p}_k$ and using $\mathbf{r}_k^T A\mathbf{p}_{k-1} = -\gamma_k/\alpha_{k-1}$ and $\mathbf{p}_{k-1}^T A\mathbf{p}_{k-1} = \gamma_{k-1}/\alpha_{k-1}$ yields

$$\beta_k = \frac{\gamma_k}{\gamma_{k-1}}, \qquad \alpha_k = \frac{\gamma_k}{\delta_k - \beta_k \gamma_k / \alpha_{k-1}}. \tag{3}$$

Both inner products are now available at the same moment and travel in one `MPI_Allreduce` of two values. The four vector updates ($\mathbf{p}, \mathbf{s}, \mathbf{x}, \mathbf{r}$) and γ are fused into one sweep and δ into the stencil sweep, so the memory traffic per iteration stays that of classic CG.

The pipelined variant of Ghysels and Vanroose [3] (`-a pipecg`) goes one step further and removes the dependency between the reduction and the stencil. It adds $\mathbf{z}_k = A\mathbf{s}_k$ and $\mathbf{q}_k = A\mathbf{w}_k$ with the recurrences $\mathbf{z}_k = \mathbf{q}_k + \beta_k \mathbf{z}_{k-1}$ and $\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha_k \mathbf{z}_k$. Now γ_k and δ_k depend only on $\mathbf{r}_k$ and $\mathbf{w}_k$, which are known before $\mathbf{q}_k$ is computed. The solver therefore starts an `MPI_Iallreduce` of (γ_k, δ_k), computes $\mathbf{q}_k = A\mathbf{w}_k$ while the reduction is in flight (polling it between stencil chunks, as for the halo), waits for the result, evaluates (3) and performs the six vector recurrences in one fused sweep that also produces the next local γ and δ. The reduction latency disappears behind the stencil. The price is three extra vectors, 36% more memory traffic, and recurrences for $\mathbf{w}$ and $\mathbf{z}$ whose rounding errors are never corrected, which limits the attainable accuracy (section 4).

3.4 Threads within a process

MPI is initialised with `MPI_THREAD_FUNNELED`: only the master thread calls MPI, always outside parallel regions. Every loop nest is an `omp parallel for collapse(2) schedule(static)` over the (i, j) index pairs with an `omp simd` loop over the contiguous k index, and the fused inner products are OpenMP reductions. Arrays are 64-byte aligned and initialised with the same static schedule that later computes on them, so on multi-socket nodes the first-touch policy places each thread's pages in its own NUMA domain. Processes and threads are pinned (`--map-by ppr:`n`:node:PE=`T `--bind-to core`, `OMP_PLACES=cores`, `OMP_PROC_BIND=close`), so the threads of one process share a socket. Fewer processes with more threads reduce the number of messages, the number of reduction participants and the ghost memory; in exchange every kernel pays a fork–join (five parallel regions per iteration with blocking halos, up to about fifteen with overlap), and the worker threads idle while the master thread waits in MPI.

3.5 Offloading to GPUs

The GPU solver (`src/cg_gpu.c`) keeps the decomposition and the MPI structure of the CPU code and replaces the threads of each process by one GPU: OpenMP is used for target offload, the "X" of MPI + X. Processes on a node take consecutive devices (`omp_set_default_device` on the node-local rank). All vectors are allocated in GPU memory once, with `omp target enter data`, and never leave it during the solve, because a single transfer of one vector over PCIe would cost more than a whole iteration on the device. Every kernel—stencil with fused inner product, fused vector updates, face packing and unpacking—is an `omp target teams distribute parallel for collapse(3)` loop over data that is already present, and inner products are OpenMP reductions on the device followed by `MPI_Allreduce` on the host. The halo exchange packs each face on the GPU, copies only the packed faces to host buffers (`omp target update`), exchanges them with `MPI_Sendrecv` and copies them back for unpacking, so the bus carries V values per iteration instead of whole vectors. Classic CG and CG1 are available. Every kernel is launched once before the clock starts, so the one-off kernel loading of the CUDA runtime is not charged to the iteration.

3.6 Performance model

The decisions above enter the time per iteration through four terms,

$$t_{\text{iter}}(P) \;\approx\; \frac{N^3}{P}\,\frac{B_v}{b(P)} \;+\; m\,t_s + V\,t_w \;+\; \rho\,\lceil \log_2 P \rceil\, t_s, \tag{4}$$

Table 1: Cost of the three CG formulations per iteration.

variant	reductions ρ	values each	vectors	B_v (B/point)
classic CG	2	1	4	88
Chronopoulos–Gear (CG1)	1	2	5	88
pipelined CG	0 (overlapped)	2	7	120

where B_v is the memory traffic per grid point of the CG variant (table 1), $b(P)$ the memory bandwidth available to one process when P processes share the hardware, m and V the message count and halo volume of the busiest process from (2), t_s the latency of a message, t_w the transfer time per value, and ρ the number of reductions per iteration that are not hidden. B_v counts explicit loads and stores in the convention of the STREAM benchmark [5], assuming that the stencil's neighbour values are reused from cache. For classic CG the stencil reads $\mathbf{p}$ and writes $\mathbf{q}$, the $\mathbf{x}, \mathbf{r}$ update reads four vectors and writes two, and the $\mathbf{p}$ update reads two and writes one: $2 + 6 + 3 = 11$ accesses, or 88 bytes. Fusing the two inner products into these sweeps avoids three further reads. On a CPU the stencil's pure write of $\mathbf{q}$ first loads the cache line (*write-allocate*), which adds 8 bytes; we therefore convert times into DRAM bandwidth with 96 bytes per point on the CPU and 88 on the GPU, whose caches write full sectors without reading them.

The model makes the scaling behaviour explicit. The memory term falls like $1/P$ only while $b(P)$ stays constant, that is, until the processes on a socket saturate its memory bandwidth; from then on adding cores on the same socket gains nothing, while adding nodes adds bandwidth. The halo term falls like $P^{-2/3}$ for blocks and not at all for slabs. The reduction term grows with $\log_2 P$ and is the only term that grows, so it eventually dominates every strong-scaling run. CG1 halves it at no cost in B_v, and pipelined CG removes it at a 36% higher B_v, which pays off once the hidden latency exceeds the extra memory time.

4. Validation

We validate the solvers at three levels: agreement with the serial code, convergence to the exact solution, and attainable accuracy. The script `scripts/validate.sh full` runs all CPU tests and `scripts/check_validation.py` evaluates them; `scripts/run_gpu.sh` repeats the first level on the GPU. All tests pass.

The first level compares every parallel configuration with the serial solver point by point. We ran all combinations of the three CG variants, the two halo modes and the three decompositions on $P \in \{1, 2, 3, 4, 6, 8\}$ processes with $T \in \{1, 2\}$ threads ($T = 2$ for $P \leq 4$) on a 40^3 grid, chosen because 40 is not divisible by 3 or 6 so that uneven blocks occur. Each run writes its solution in global order with collective MPI-IO (`MPI_Type_create_subarray` and `MPI_File_write_all`), and the file is compared with the serial solution. Table 2 summarises the 180 runs. Classic CG and CG1 agree with the serial code to 3.8×10^{-15} relative to the solution's maximum, which is rounding level and stems only from the different summation order of the distributed inner products, and they need exactly the serial number of iterations (146). Pipelined CG agrees to 2.0×10^{-12}, the effect of its extra recurrences and still seven orders of magnitude below the discretisation error (2.39×10^{-5}). A further 30 runs cover the corner cases of the shell decomposition of section 3.2—one-plane slabs, two-point blocks, $N = 7$ on six processes and $N = 2$ on two—and all reproduce the serial solution to rounding level. On Vanda the quick suite (36 variant runs, 12 edge cases and the grid study) ran on a 32-core Intel(R) Xeon(R) Platinum 8452Y node and passes with the same worst differences, 3.8×10^{-15} and 2.0×10^{-12}. The GPU solver ran CG and CG1 on $P = 1, 2, 4, 8$ processes sharing one A40, so that the $P = 8$ runs exchange halos in all three directions through device packing, host staging and MPI; all 8 runs need 146 iterations and agree with the serial solution to 4.0×10^{-15}.

Table 2: Parallel against serial solution on the 40^3 grid. "max diff" is $\max |x_{\text{par}} - x_{\text{ser}}| / \max |x_{\text{ser}}|$ over all runs of a row; "true residual" is the largest $\|\mathbf{b} - A\mathbf{x}\|/\|\mathbf{b}\|$ at tolerance $\varepsilon = 10^{-8}$.

algorithm	halo	runs	P	T	iterations	max diff	true residual
CG	blocking	30	1–8	1–2	146–146	3.5×10^{-15}	8.24×10^{-9}
CG	overlap	30	1–8	1–2	146–146	3.1×10^{-15}	8.24×10^{-9}
CG1	blocking	30	1–8	1–2	146–146	3.8×10^{-15}	8.24×10^{-9}
CG1	overlap	30	1–8	1–2	146–146	3.8×10^{-15}	8.24×10^{-9}
pipelined CG	blocking	30	1–8	1–2	146–146	2.0×10^{-12}	8.24×10^{-9}
pipelined CG	overlap	30	1–8	1–2	146–146	2.0×10^{-12}	8.24×10^{-9}

The second level checks that the solver computes the solution of the differential equation and not only of some linear system. Table 3 and fig. 2(a) report the discrete L^2 error $\|u_h - u\|_{L^2} = \left(h^3 \sum_{ijk} (u_{ijk} - u(x_i, y_j, z_k))^2\right)^{1/2}$

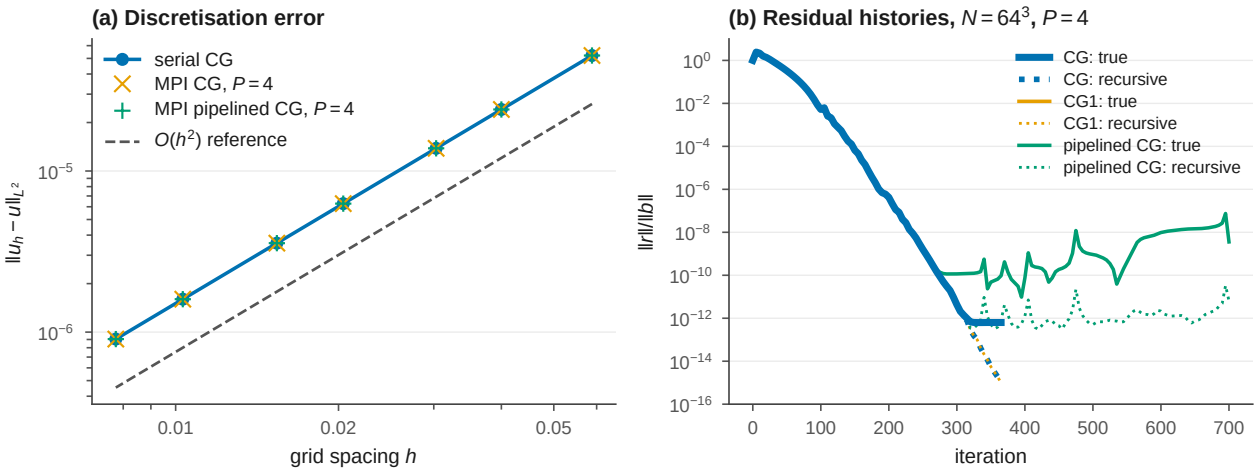


Figure 2: (a) Discretisation error against grid spacing for the serial and parallel solvers, with an $O(h^2)$ reference line. (b) True (solid) and recursively updated (dotted) relative residuals, $N = 64$, $P = 4$; the CG and CG1 curves coincide.

for $N = 16$ to 128, solved to $\varepsilon = 10^{-10}$ so that the algebraic error is negligible. The observed order is 2.000, exactly the order of the 7-point scheme, and the parallel errors match the serial ones in every printed digit.

Table 3: Grid convergence of the serial solver and of the parallel solver ($P = 4$, blocks, overlapped halo).

N	h	iterations	$\|u_h - u\|_{L^2}$ serial	$\|u_h - u\|_{L^2}$ parallel	observed order
16	5.88×10^{-2}	68	5.210×10^{-5}	5.210×10^{-5}	–
24	4.00×10^{-2}	102	2.410×10^{-5}	2.410×10^{-5}	1.999
32	3.03×10^{-2}	136	1.383×10^{-5}	1.383×10^{-5}	2.000
48	2.04×10^{-2}	205	6.274×10^{-6}	6.274×10^{-6}	2.000
64	1.54×10^{-2}	274	3.565×10^{-6}	3.565×10^{-6}	2.000
96	1.03×10^{-2}	413	1.601×10^{-6}	1.601×10^{-6}	2.000
128	7.75×10^{-3}	552	9.052×10^{-7}	9.052×10^{-7}	2.000

The third level probes the numerical price of the reformulations. Figure 2(b) runs the three variants towards a tolerance of 10^{-15} and records both the recursively updated residual, which the algorithm sees, and the true residual $\|\mathbf{b} - A\mathbf{x}_k\|$, computed with an extra untimed stencil application. CG and CG1 are indistinguishable, and their true residual levels off at 6×10^{-13} once rounding errors in $\mathbf{x}$ dominate. Pipelined CG follows them until the two residuals separate near 3×10^{-11}; its best true residual is 1×10^{-11}, 15 times worse than CG, and when the iteration continues its recursive residual stagnates while the true residual drifts back up to 4×10^{-9} after 700 iterations. This is the residual gap of pipelined Krylov methods: latency hiding costs numerical robustness. On the 40^3 validation grid all three variants still meet $\varepsilon = 10^{-8}$ (table 2), but the gap grows with the problem: on the 512^3 grid of section 5.7 the true residual of pipelined CG ends at 70 times the tolerance its recursive residual reports. Production use of the pipelined variant therefore needs residual replacement, i.e. periodically recomputing $\mathbf{r} = \mathbf{b} - A\mathbf{x}$ and $\mathbf{w} = A\mathbf{r}$ explicitly.

5. Performance

Setup. All measurements were taken on the NUS Vanda cluster through PBS Pro. A Vanda CPU node carries two Intel(R) Xeon(R) Platinum 8452Y processors with 36 cores each and two NUMA domains. The multi-node job requested 8 chunks of 32 cores on 8 different nodes; on every node PBS assigned the same 32 odd-numbered cores, all on one socket, so each node contributes exactly one socket and its memory controllers, and the job is a clean model of a cluster of 32-core nodes. MPI processes on remote nodes were started through PBS (`pbs_tmrsh`), which keeps them inside the job's CPU set, and every process was pinned to its own core. The code was built with GCC 13.1.0 (`-O3 -march=native -fopenmp`) and run with Open MPI 4.1.8. Strong-scaling runs use $N = 512$ (134 million unknowns, four to seven vectors of 1.07 GB) and $N = 768$ (453 million unknowns), weak-scaling runs 192^3 points per process. Each timing run performs exactly 50 iterations of the steady-state loop; work vectors are allocated before the clock starts, and the one-off setup stencil application of CG1 and pipelined CG counts in the time to solution but not in the time per iteration. Every configuration ran 3 times, interleaved with the others, and we report the fastest run. The median spread between repetitions is 3.7%, but the nodes are shared with other users'

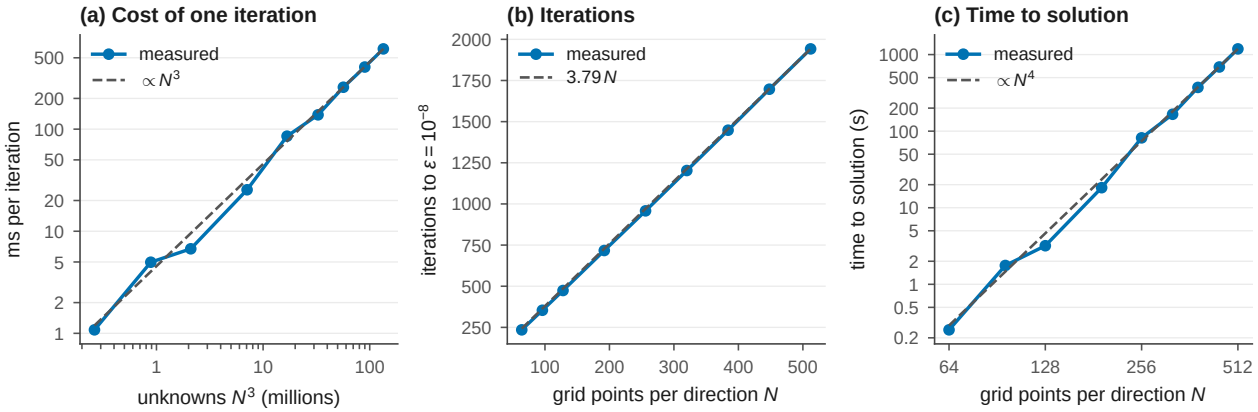


Figure 3: Serial solver on one core of a Intel(R) Xeon(R) Platinum 8452Y node. (a) Time per iteration against the matrix dimension; (b) iterations to $\varepsilon = 10^{-8}$; (c) time to solution. Dashed lines are the expected power laws.

Table 4: Serial problem-size study: unknowns and nonzeros (millions), memory of the five vectors (MiB), time per iteration (best of three 20-iteration runs and the complete solve), cost per grid point, effective DRAM bandwidth (96 B/point), iterations and time to $\varepsilon = 10^{-8}$, and the maximum error against the exact solution.

N	unknowns	nonzeros	MiB	ms/iter	ns/point	GB/s	iterations	solve (s)	max error
64	0.26	1.8	10	1.08	4.13	23.3	235	0.3	9.53×10^{-6}
96	0.88	6.1	34	4.97	5.62	17.1	354	1.8	4.28×10^{-6}
128	2.10	14.6	80	6.73	3.21	29.9	474	3.2	2.42×10^{-6}
192	7.08	49.3	270	25.48	3.60	26.7	717	18.3	1.08×10^{-6}
256	16.78	117.0	640	85.36	5.09	18.9	958	81.8	6.10×10^{-7}
320	32.77	228.8	1250	138.10	4.21	22.8	1203	166.1	3.91×10^{-7}
384	56.62	395.5	2160	257.36	4.55	21.1	1448	372.7	2.72×10^{-7}
448	89.92	628.2	3430	405.87	4.51	21.3	1697	688.8	2.00×10^{-7}
512	134.22	938.0	5120	611.07	4.55	21.1	1942	1186.7	1.53×10^{-7}

jobs, and a few multi-node runs caught a slow node or a busy network and took up to 166% longer; the fastest run is the one that measures the code. The GPU runs used one NVIDIA A40 (45 GB, driver 575.57.08) on a Vanda GPU node, compiled with `nvc` 26.1-0 of the NVIDIA HPC SDK (`-O3 -mp=gpu -gpu=cc86`).

5.1 Serial baseline and problem size

Figure 3 and table 4 run the serial solver on one core for $N = 64$ to 512, so the matrix dimension grows from 0.26 to 134 million and the number of nonzeros from 1.8 to 938 million. The time per iteration grows like $N^{3.05}$, i.e. linearly in the number of unknowns, as expected for a matrix-free method whose every kernel is a sweep over the vectors. The cost per grid point stays between 3.21 and 5.62 ns over the whole range, with no step where the five vectors (10 MB at $N = 64$, 5 GB at $N = 512$) outgrow the 67 MB L3 cache of a socket: one core draws 17–30 GB/s whether the data comes from L3 or from DRAM, because its bandwidth is limited by the number of cache misses it can keep in flight, not by the memory level (section 5.2). The iteration count grows linearly in N, from 235 to 1942 iterations ($\approx 3.79\,N$), so the time to solution grows like $N^{4.06}$ and reaches 1187 s, 19.8 minutes, on the 512^3 grid. Doubling the resolution makes the solve sixteen times more expensive; this is the work that the parallel solver has to distribute. The size study ran on one core of a node shared with other jobs (611 ms per 512^3 iteration); the serial baseline for the speedups below was measured inside our 32-core chunk, whose other cores were idle, at 546 ms.

5.2 Strong scaling from one core to 8 nodes

Figure 4 and table 5 show the baseline configuration: pure MPI, blocks, overlapped halo and classic CG, 32 processes per node. The parallel code on one process is 1.2% faster than the serial code, so the parallel machinery costs nothing. The curve has two regimes separated by the node boundary.

Inside one socket the speedup is ideal up to four processes (3.91, 98% efficiency) and then flattens: 5.8 on eight processes and 9.1 on 32, an efficiency of 28%. Communication does not explain this, because halo exchange and reductions take 5% of the iteration at $P = 32$. Figure 5 converts the time per iteration into DRAM traffic with the 96-byte model of section 3.6 and compares it with the STREAM triad counted the same way. One core moves 23.9 GB/s, more than STREAM on one core (20.0 GB/s), whose single stream of prefetches draws less than the solver's several concurrent streams. STREAM grows linearly to eight threads and then saturates, and the solver

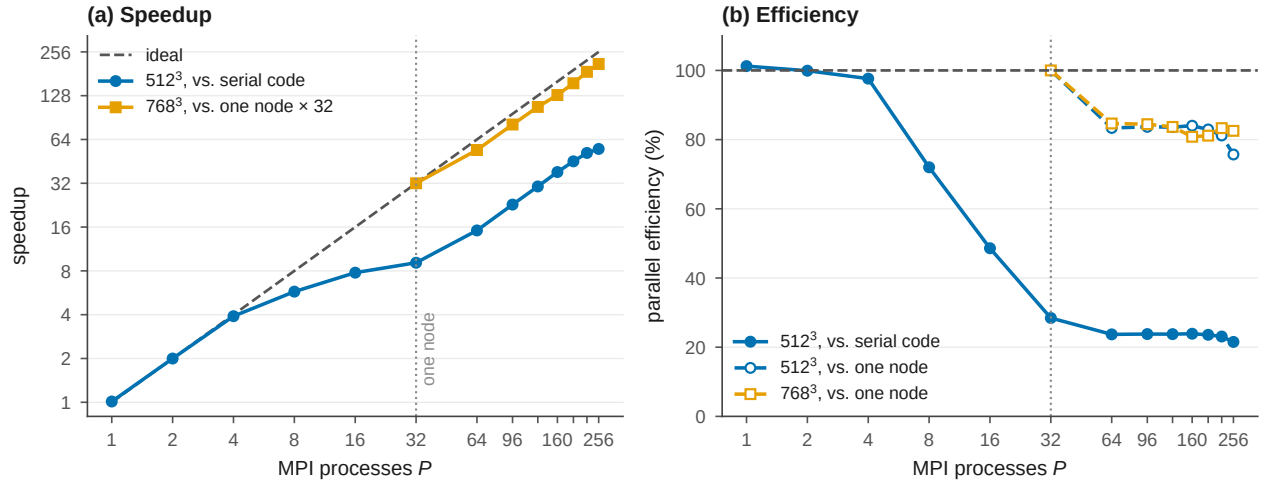


Figure 4: Strong scaling of the baseline (pure MPI, 3D blocks, overlapped halo, classic CG), 32 processes per node. 512^3: speedup and efficiency relative to the serial code (solid) and relative to one node (dashed, $32\,t_{32}/(P\,t_P)$); 768^3: relative to one node.

Table 5: Strong scaling, 50 iterations. "eff" is $S(P)/P$ relative to the serial code, "node eff" relative to one node; "comm" is the share of the iteration spent in halo exchange and reductions, including waiting; "halo" is the data sent by the busiest process per iteration.

			$N = 512^3$						$N = 768^3$	
P	nodes	process grid	ms/iter	speedup	eff (%)	node eff (%)	comm (%)	halo (KiB)	ms/iter	node eff (%)
1	1	1×1×1	539.77	1.0	101	–	0	0	–	–
2	1	2×1×1	273.45	2.0	100	–	2	2048	–	–
4	1	2×2×1	139.93	3.9	98	–	3	2048	–	–
8	1	2×2×2	94.87	5.8	72	–	5	1536	–	–
16	1	4×2×2	70.27	7.8	49	–	6	1536	–	–
32	1	4×4×2	60.06	9.1	28	100	5	1152	201.6	100
64	2	4×4×4	36.02	15.2	24	83	20	768	119.1	85
96	3	6×4×4	23.92	22.8	24	84	21	600	79.6	84
128	4	8×4×4	17.96	30.4	24	84	21	512	60.2	84
160	5	8×5×4	14.30	38.2	24	84	22	437	49.9	81
192	6	8×6×4	12.07	45.3	24	83	25	386	41.4	81
224	7	8×7×4	10.57	51.7	23	81	27	347	34.6	83
256	8	8×8×4	9.92	55.1	22	76	40	320	30.5	83

follows the same curve to 215 GB/s on 32 cores, 77% of the socket's STREAM bandwidth (279 GB/s). Cores added after the memory controllers saturate have nothing to compute on: inside a socket the speedup of CG is the bandwidth speedup.

Across nodes the picture reverses, because every node adds a socket's worth of memory bandwidth. Going from one node to two cuts the iteration from 60.1 to 36.0 ms (83% node-level efficiency), and 8 nodes reach 9.92 ms, 6.06 times faster than one node (76% efficiency) and 55.1 times faster than the serial code. The aggregate traffic rises to 1299 GB/s, 1.30 TB/s, close to the dotted line of fig. 5 that multiplies the bandwidth of one socket by the number of nodes. On the larger 768^3 grid, where each process owns 3.4 times more points but sends only 2.25 times more halo data, 8 nodes are 6.60 times faster than one (201.6 to 30.5 ms), 83% efficiency. The efficiency that is lost across nodes is lost to communication, which section 5.3 dissects.

5.3 Where the communication time goes

The breakdown in fig. 7 splits each iteration into its four phases. On one process the vector updates take 56% of the time, more than the stencil, because they account for nine of the eleven memory accesses per point while doing almost no arithmetic. Inside the node the communication phases stay thin. Beyond it they grow with every node, and at $P = 256$ they take 40% of the iteration.

The two kinds of communication behave very differently. The halo exchange is cheap: at $P = 256$ the busiest process sends 320 KiB per iteration in six messages, which the network delivers in a few tens of microseconds, and

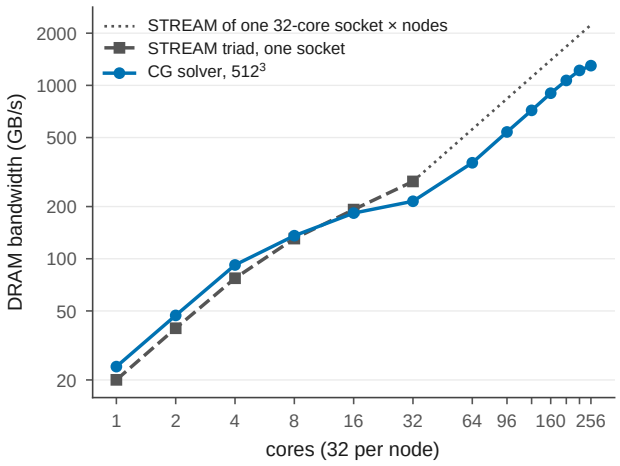


Figure 5: DRAM traffic of the 512^3 baseline (96 bytes per point and iteration) against the STREAM triad of one socket (32 bytes per element); the dotted line multiplies the socket's STREAM bandwidth by the number of nodes.

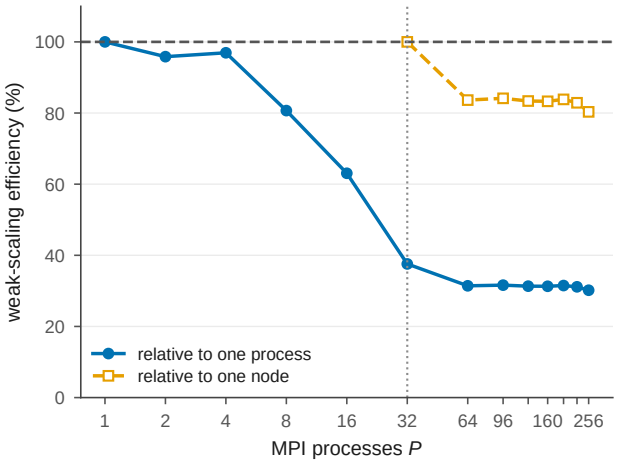


Figure 6: Weak scaling with $\approx 192^3$ points per process: time per grid point and iteration relative to one process and relative to one full node.

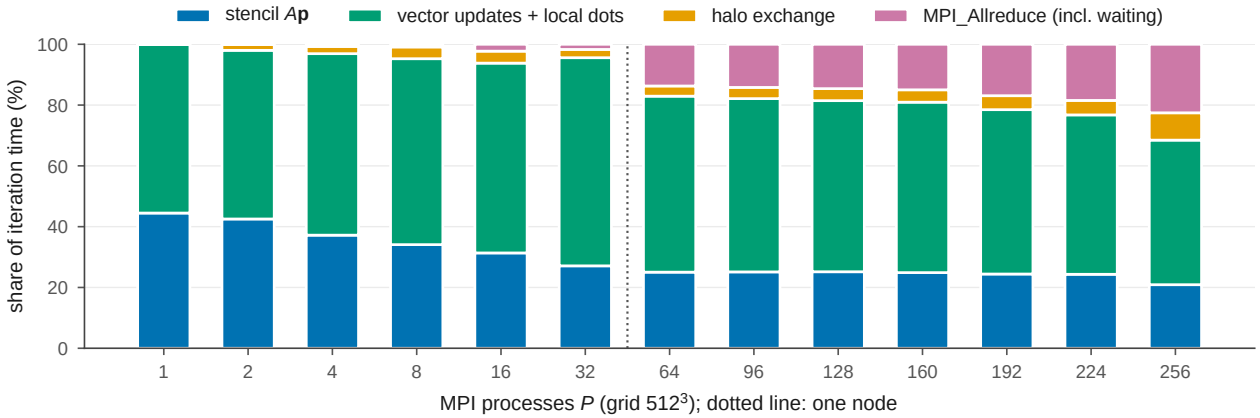


Figure 7: Share of the iteration time spent in the stencil, the vector updates, the halo exchange and the reductions (each phase timed as the maximum over processes; the reduction time includes waiting for slower processes; absolute times in table 5).

the whole phase, including face packing and unpacking, costs 1.12 ms or 11% of the iteration. The reductions are expensive: they take 2.81 ms, 28% of the iteration, although the two `MPI_Allreduce` calls move 16 bytes and a reduction over 256 processes needs only about $\lceil \log_2 256 \rceil = 8$ network latencies, a few tens of microseconds. The rest is waiting. Every process has the same amount of work (the $8 \times 8 \times 4$ process grid divides 512^3 evenly), but every process also shares its socket's memory bandwidth with 31 others and its node with other jobs, so in each iteration some process finishes its sweep late, and the reduction makes all 256 processes wait for it. On one node this barrier effect costs 1.04 ms per iteration; across 8 nodes, where the slowest of eight sockets sets the pace, it costs 2.81 ms. The global reductions, not the halo messages, are what limits strong scaling across nodes, and they act as barriers far more than as messages.

Weak scaling (fig. 6) separates the two regimes cleanly. With 192^3 points per process one process completes an iteration in 37.4 ms, drawing 18.1 GB/s. Thirty-two processes on one socket need 100.5 ms for 32 times the data, because they share the bandwidth (6.8 GB/s each); the efficiency relative to one process falls to 38%, the bandwidth wall again. From one node to 256 processes on 8 nodes, where each node brings its own memory, the time grows only from 100.5 to 125.1 ms while the problem grows to 1219^3, 1.81 billion unknowns: a node-level weak-scaling efficiency of 80%. The missing fifth is the halo and reduction cost of (4). Weak scaling of the time per iteration also flatters the algorithm: the iteration count grows like $N \propto P^{1/3}$, so the time to solution grows by the same factor even when every iteration scales perfectly, a growth that only a better preconditioner removes.

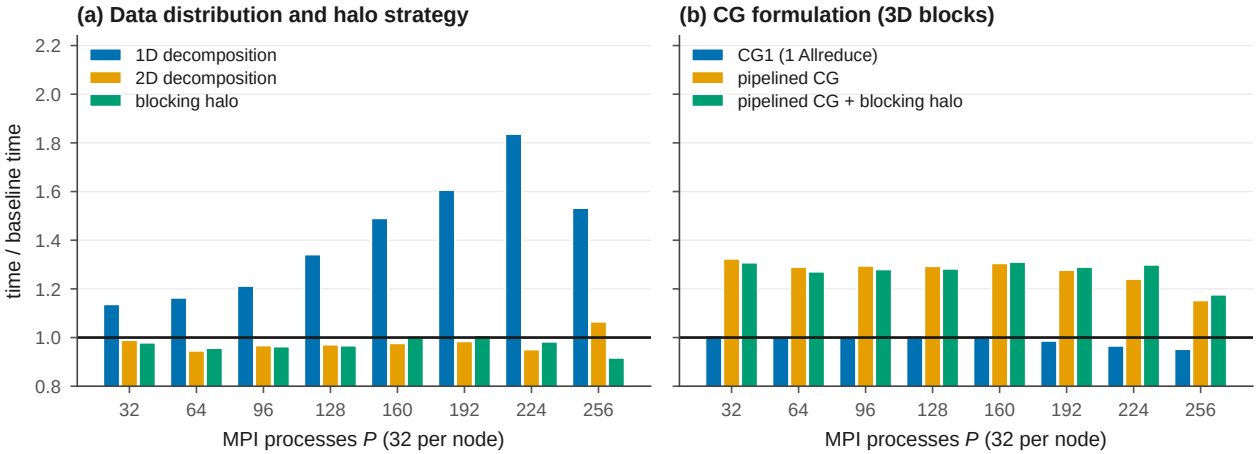


Figure 8: Time per iteration of each variant divided by the baseline (3D blocks, overlapped halo, classic CG) at the same P, $N = 512^3$, 32 processes per node; values below 1 are faster.

Table 6: Time per iteration in ms and, in parentheses, relative to the baseline; $N = 512^3$, one, two, four and eight nodes (fig. 8 shows all process counts).

variant	$P = 32$	$P = 64$	$P = 128$	$P = 256$
3D, overlap, CG (baseline)	60.06	36.02	17.96	9.92
1D decomposition	68.24 (1.14)	41.89 (1.16)	24.08 (1.34)	15.19 (1.53)
2D decomposition	59.40 (0.99)	34.05 (0.95)	17.43 (0.97)	10.56 (1.06)
blocking halo	58.76 (0.98)	34.44 (0.96)	17.35 (0.97)	9.09 (0.92)
CG1 (1 Allreduce)	60.59 (1.01)	36.19 (1.00)	18.10 (1.01)	9.44 (0.95)
pipelined CG	79.48 (1.32)	46.45 (1.29)	23.21 (1.29)	11.43 (1.15)
pipelined CG + blocking halo	78.52 (1.31)	45.75 (1.27)	23.02 (1.28)	11.66 (1.18)

5.4 Effect of the design choices across nodes

Figure 8 and table 6 vary one decision at a time around the baseline on one to 8 nodes and report the time per iteration relative to the baseline at the same P.

The decomposition matters most, and the volume model of eq. (2) predicts it. Slabs are 1.14 times slower than blocks on one node and 1.53 times slower on 8 nodes: every slab process ships two full 512^2 planes, 4 MiB per iteration whatever P, and at $P = 256$ each slab is only two planes thick, so a process sends as much data as it owns. Pencils stay within a few percent of blocks (0.95–1.06): their halo volume falls like $P^{-1/2}$, which at these process counts is close enough to the $P^{-2/3}$ of blocks, and they avoid the strided k-faces.

Overlapping the halo exchange does not pay, even across the network. The blocking exchange runs at 0.92–1.01 times the baseline and is 8% faster on 8 nodes (0.92). The reason is the one found in section 5.3: the messages are small and the network fast, so there is little transfer time to hide, while the overlapped mode pays for the chunked interior sweep, the polling and the second pass over the boundary shell. Overlap would pay with larger faces per process or a slower network.

The CG formulations confirm that the reductions are the cost that grows. CG1 merges the two reductions into one. On one node, where they cost little, it runs at 1.01 times the baseline; on 8 nodes it is 5% faster (0.95), because each iteration now passes one barrier instead of two. Pipelined CG pays its 36% traffic surcharge almost in full on one node (1.32, model 1.36), and its relative cost falls to 1.15 on 8 nodes, where more reduction time is there to hide. It does not break even. The model of section 3.6 would let it win once the hidden reduction time exceeds 36% of the memory time, and at $P = 256$ the two are comparable, but the `MPI_Iallreduce` hides the latency of the reduction, not the waiting for the slowest process, which still arrives after the overlapped stencil. On the larger 768^3 grid, with less communication per byte of memory traffic, pipelined CG remains 1.28 times slower on 8 nodes.

5.5 Processes versus threads on 256 cores

Figure 9 and table 7 keep all 256 cores busy and vary the split into MPI processes and OpenMP threads, from 256 single-threaded processes to eight processes with 32 threads, one per node. All splits lie within 9% of each other, so the hybrid code loses nothing to its threads. The best split is 32×8 (8.72 ms), 6% faster than pure MPI (9.23 ms): four processes per socket mean eight times fewer participants in each reduction and fewer, larger halo

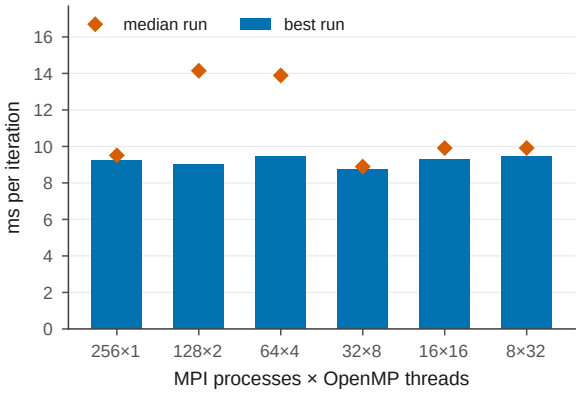


Figure 9: Time per iteration for different process × thread splits of the same 256 cores, 512^3.

Table 7: Process × thread splits on 256 cores: best and median time per iteration and the halo and reduction parts of the best run (ms, including waiting).

P	T	best	median	halo	reduce
256	1	9.23	9.51	0.52	2.07
128	2	9.02	14.15	0.40	1.84
64	4	9.45	13.89	0.54	2.29
32	8	8.72	8.89	0.38	1.51
16	16	9.28	9.91	0.30	1.70
8	32	9.46	9.91	0.19	1.42

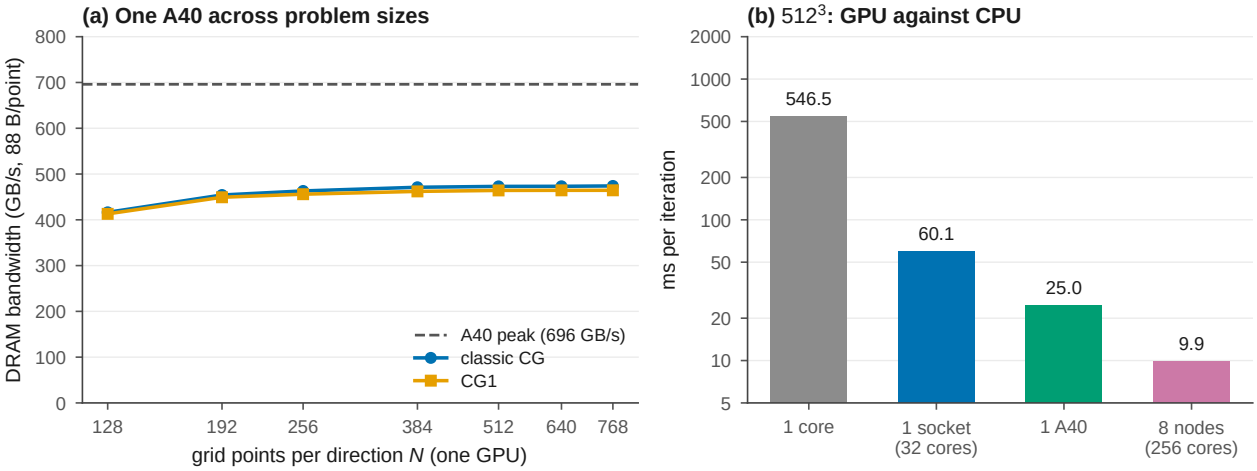


Figure 10: (a) DRAM bandwidth of the offload solver on one NVIDIA A40 (88 bytes per point and iteration). (b) Time per iteration on the 512^3 grid: serial code, one 32-core socket, one A40 and eight nodes.

messages, and the reduction wait drops from 2.07 to 1.51 ms per iteration. With more threads per process the fork–join cost of the many parallel regions per iteration grows, and one process per node with 32 threads (9.46 ms) is the slowest split. The median runs (diamonds) show that the splits with 64 and 128 processes also suffered most from slow repetitions on the shared nodes. The right configuration is a few processes per socket with eight threads each.

5.6 GPU offload

Figure 10 and table 8 run the offload solver on one A40 from $N = 128$ to $N = 768$, the largest grid holding 453 million unknowns. Above 128^3 the GPU sustains a constant 416–474 GB/s of DRAM traffic, 68% of its nominal 696 GB/s; at 128^3 the grid is too small to fill the device and the launch overhead of the three kernels per iteration shows. The stencil takes 31% of an iteration and the two vector-update kernels the rest; both run at memory speed, 415 and 499 GB/s. CG1 is 2% slower than classic CG on the GPU: its fused five-vector kernel saves no traffic, and there are no reductions to merge on a single device.

On the 512^3 grid one A40 needs 25.0 ms per iteration and moves 473 GB/s. That is 2.4 times faster than a 32-core CPU socket, 22 times faster than one core and 2.5 times slower than the 8 nodes of section 5.2; the ratio to the socket is the ratio of the memory bandwidths, as the model predicts. With several GPUs every halo message passes through host memory twice, so the communication terms of (4) weigh more than on the CPU; the validation runs of section 4 exercise this path on up to eight processes.

5.7 Time to solution

Table 9 closes the loop with complete solves of the 512^3 problem to $\varepsilon = 10^{-8}$. All runs stop after 1942 iterations and reach the same error against the exact solution, which at this tolerance is set by the discretisation. The serial code needs 1187 s. 256 cores on 8 nodes solve the problem in 21.0 s with classic CG and 20.6 s with CG1, 56 times faster than the serial code; one A40 needs 48.5 s. Pipelined CG takes 25.5 s and is the only variant whose answer

Table 8: One A40: unknowns (millions), memory of the four CG vectors (GiB), time per iteration of classic CG and CG1 (best of 3 runs of 50 iterations), DRAM bandwidth, share of the stencil kernel, and billions of grid-point updates per second.

N	unknowns	GiB	CG ms/iter	CG1 ms/iter	GB/s	stencil (%)	GLUP/s
128	2	0.1	0.44	0.45	416	32	4.73
192	7	0.2	1.37	1.39	454	31	5.16
256	17	0.5	3.19	3.24	463	31	5.26
384	57	1.7	10.58	10.78	471	31	5.35
512	134	4.0	24.97	25.44	473	31	5.37
640	262	7.8	48.75	49.67	473	31	5.38
768	453	13.5	84.10	85.83	474	31	5.39

Table 9: Complete solves to $\varepsilon = 10^{-8}$, $N = 512^3$ (CPU runs: blocks, overlapped halo, 32 processes per node). "speedup" is relative to the serial solve.

platform	algorithm	iterations	time (s)	ms/iter	true residual	max error	speedup
1 core	CG	1942	1187	611	9.76×10^{-9}	1.53×10^{-7}	1
256 cores (8 nodes)	CG	1942	21.0	10.83	9.76×10^{-9}	1.53×10^{-7}	56
256 cores (8 nodes)	CG1	1942	20.6	10.62	9.76×10^{-9}	1.53×10^{-7}	58
256 cores (8 nodes)	pipelined CG	1942	25.5	13.13	6.96×10^{-7}	1.53×10^{-7}	47
1 A40 GPU	CG	1942	48.5	24.98	9.76×10^{-9}	1.53×10^{-7}	24
1 A40 GPU	CG1	1942	49.4	25.44	9.76×10^{-9}	1.53×10^{-7}	24

is worse: its true residual ends at 7.0×10^{-7}, 70 times the tolerance its recursive residual reports. The residual gap of section 4 grows with the problem and is the second cost of hiding the reduction, next to the extra memory traffic.

6. Conclusions

We built a matrix-free Conjugate Gradient solver for the 3D Poisson equation in a serial, a hybrid MPI + OpenMP and an MPI + OpenMP offload version, in which decomposition, halo exchange, reduction strategy and threading are run-time switches. All versions reproduce the serial solution to rounding level and converge to the exact solution at the theoretical second order, so every performance difference we measure is a property of the parallel design.

The problem is parallelised by distributing the grid in 3D blocks with one ghost layer, so that each process applies the stencil to its own block after a six-message halo exchange, and by turning each inner product into a local OpenMP or GPU reduction followed by a global `MPI_Allreduce`. Its performance is set by memory bandwidth first and by the reductions second. Inside a socket the solver scales exactly as far as the memory controllers do, 77% of STREAM on 32 cores and a speedup of 9.1. Across nodes each socket adds bandwidth, and 256 cores on 8 nodes reach a speedup of 55.1 over the serial code, 76% strong-scaling efficiency relative to one node on 512^3, 83% on 768^3 and 80% weak-scaling efficiency on 1.8 billion unknowns.

The part of the algorithm that communicates most in time is not the part that communicates most in bytes. The halo exchange moves all of the data—320 KiB per process and iteration at $P = 256$—and costs 11% of the iteration; the two reductions move 16 bytes and cost 28%, because every reduction is a barrier at which 256 processes wait for the slowest one. The design choices follow from this. Blocks or pencils are mandatory, since slabs send as much as they compute and run 1.53 times slower on 8 nodes. Overlapping the halo exchange hides little and costs more. Merging the two reductions (CG1) costs nothing and saves 5% at scale, so it is the formulation we recommend. Pipelining the reduction hides its latency but not the waiting, adds 36% memory traffic and degrades the true residual 70-fold, so it does not pay on this machine. A few MPI processes per socket with eight OpenMP threads each cut the reduction wait and give the fastest split on 256 cores. One A40 GPU delivers the bandwidth of 2.4 CPU sockets and solves the 512^3 problem in 48.5 s.

The next steps follow from the same analysis: a multigrid or Chebyshev preconditioner, which removes the $O(N)$ growth of the iteration count, and communication-avoiding s-step methods, which perform one reduction per s iterations and attack the barrier cost directly.

Code and data. The source code, the PBS job scripts, the scripts that produce every figure and table, the raw CSV measurements and job logs, and a `README.md` with full run instructions are open source (MIT licence) at

https://github.com/davidzuo023/dsa5208-project3 [7].

Use of AI tools. We used Claude (Anthropic) as a coding and writing assistant. Under our direction it wrote most of the implementation, the test and benchmark scripts and a first draft of this report. We chose the problem and the parallelisation questions, obtained access to Vanda and ran all jobs there, checked the results against the validation suite, and reviewed and edited the report. All numbers in this report come from our runs; the commit history of the repository reflects this workflow.

References

1. M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *J. Res. Nat. Bur. Standards* 49:409–436, 1952.
2. A. T. Chronopoulos and C. W. Gear. s-step iterative methods for symmetric linear systems. *J. Comput. Appl. Math.* 25:153–168, 1989.
3. P. Ghysels and W. Vanroose. Hiding global synchronization latency in the preconditioned Conjugate Gradient algorithm. *Parallel Computing* 40:224–238, 2014.
4. J. Dongarra, M. A. Heroux and P. Luszczek. High-performance conjugate-gradient benchmark: a new metric for ranking high-performance computing systems. *Int. J. High Perform. Comput. Appl.* 30(1):3–10, 2016.
5. J. D. McCalpin. Memory bandwidth and machine balance in current high performance computers. *IEEE TCCA Newsletter*, 1995.
6. J. R. Shewchuk. *An Introduction to the Conjugate Gradient Method Without the Agonizing Pain*. Carnegie Mellon University, 1994.
7. Y. Wu, D. Zuo and C. Zhu. *dsa5208-project3: Conjugate Gradients for the 3D Poisson equation with MPI, OpenMP and GPU offload*. Source code, 2026. https://github.com/davidzuo023/dsa5208-project3.